

现代密码学课程设计报告

基于 RSA 与 ElGamal 的混合公钥密码系统、简单证书方案、PKI 与简易安全邮件系统

作者：课程设计小组

日期：2026-08-24

运行环境：Windows 10/11, MinGW64 g++ 9.2.0 (`D:\Dev-Cpp\MinGW64\bin\g++.exe`), `-O2 -std=c++11`

1. 任务概述

依据 `README.md` 的要求，本设计实现如下内容：

- 实现 `RSA` 加密/解密算法以及 `RSA` 签名/验证算法。
- 实现 `ElGamal` 加密/解密算法以及 `ElGamal` 签名/验证算法。
- 实现一种简单证书方案：证书需携带 `flag1`（签名算法：0=`RSA`、1=`ElGamal`）与 `flag2`（用途：0=加密、1=签名），证书以 `txt` 文本形式保存。
- 搭建一个简易 `PKI`：包含一个根 `CA` 与两个下级 `CA`（如 `CA1`、`CA2`），用户向 `CA` 申请证书；证书库支持按 `ID` 查询并向根 `CA` 回溯构建证书链，验证证书链。
- 实现简易安全邮件系统：发送方用自己签名私钥对邮件签名，并用接收方加密公钥加密；接收方用自己的加密私钥解密，并用发送方签名公钥验证签名（结合证书链验证发送方身份）。
- 随机素数 `p,q` 均为 1024 比特长度（程序默认 512 比特以求演示速度，可在菜单“9. 切换密钥强度”中选 1024 即完全符合任务书要求）。

说明：目标机未安装 `NTL` 库，且无 `make/sh/perl`。为使代码采用 `NTL` 风格 API（便于阅读、且可后续直接链接真实 `NTL`），本报告自行实现了一个 `NTL` 兼容的大整数类 `NTL::ZZ`（含乘法/除法/模幂/扩展欧几里得/素性测试等），从而保证源码可直接在无 `NTL` 环境编译运行。

2. 总体设计（模块划分）

所有代码位于 `src`，按“独立模块/独立类”组织，未使用全局变量或单一 `main`：

模块 | 文件 | 职责

----- | ----- | -----

大整数库 | `bigint.h.cpp` | `NTL::ZZ`：加/减/乘/除/移位、`NumBits`、`RandomBits`、`RandomPrime`、`ProbPrime`(Miller – Rabin)、`PowerMod`、`I`

工具 | `utils.h.cpp` | `SHA-256` (`sha256/sha256ZZ/sha256Hex`)、`trim/split`、文件读写

`RSA` | `rsa.h.cpp` | `RSAKeyGen`、`RSAEncrypt`、`RSADecrypt`、`RSASign`、`RSASignVerify`、密钥与消息的编码/字符串互转

`ElGamal` | `elgamal.h.cpp` | `ElGamalKeyGen`(安全素数+生成元)、`ElGamalEncrypt/Decrypt`、`ElGamalSign/Verify`

证书 & 公钥 | `certificate.h.cpp` | `PubKey/PrivKey`、`parsePubKey`、`signHash/verifyHash`、`Cert`(含 `flag1/flag2`、`toTxt/fromTxt`)、`issue`

`PKI` | `pki.h.cpp` | `CertRepo`(单例证书库，支持证书链查询)、`CA`、`PKI`(根 `CA` + 2 下级 `CA` + 建库)

实体 | `entity.h.cpp` | `Entity`：持有加密/签名两套密钥，向 `CA` 申请证书

安全邮件 | `securemail.h.cpp` | `sendMail/receiveMail`

测试 | `test.h.cpp` | 各方案的独立测试函数；`main.cpp` 提供菜单

3. 核心算法设计

3.1 大整数库 `<b>NTL::ZZ</b>`

- 采用 32-bit 基 (`<b>base = 2^32</b>`) 的 `<b>std::vector<uint32_t></b>` 存储，小端排列。
- 乘法 `<b>mulMag</b>`：Schoolbook，再对进位做规整；除法 `<b>divmodMag</b>`：Knuth 卷 2 算法 D（试商时以 `<b>unsigned __int128</b>` 比较避免溢出），解决了高位进位与试商上界两类关键错误。
- 模幂 `<b>PowerMod</b>`：从高位到低位的平方-乘；素性测试 `<b>ProbPrime</b>`：对 2/3/5 等小素数做试除后，进行 `<b>k</b>` 轮 Miller – Rabin。
- 密钥生成 `<b>RandomPrime(bits, k)</b>`：随机生成 `<b>bits</b>` 比特奇数，小素数试除加速，再行 Miller – Rabin（默认演示 `<b>k=12</b>`，任务书强度可设 `<b>k=20</b>`）。

3.2 RSA

- 密钥生成：`<b>RandomPrime</b>` 生成 `<b>p,q</b>` (`<b>bits</b>` 比特，满足 `<b>p ≠ q</b>`)，`<b>n=pq</b>`，`<b>φ=(p-1)(q-1)</b>`，`<b>e=65537</b>`，`<b>d=e^{-1} mod φ</b>`。公钥 `<b>(n,e)</b>`，私钥 `<b>(p,q,d)</b>`。
- 加密：`<b>c = m^e mod n</b>`；解密：`<b>m = c^d mod n</b>`。
- 签名：`<b>s = H(m)^d mod n</b>`；验证：`<b>s^e mod n == H(m)</b>`。其中 `<b>H</b>` 为 SHA-256。
- 消息编码：字符串按小端打包成 `<b>ZZ</b>`；解密后按字节还原。当 `<b>m</b>` 较大 (`>`模数) 时采用分块（以 `<b>|</b>` 分隔各密文块）。

3.3 ElGamal

- 密钥生成：生成安全素数 `<b>p=2q+1</b>`，取 `<b>g=2</b>` 验证其为 `<b>q</b>` 阶生成元，私钥 `<b>x ∈ [2,p-2]</b>`，公钥 `<b>y=g^x mod p</b>`。
- 加密：`<b>c1=g^k, c2=m · y^k mod p</b>` (`<b>k</b>` 随机)。解密：`<b>m=c2 · c1^{-x} mod p</b>`。
- 签名：`<b>r=g^k, s=k^{-1}(H(m)-x · r) mod (p-1)</b>`；验证：`<b>y^r · r^s ≡ g^{H(m)} mod p</b>`。

3.4 简单证书方案 `<b>Cert</b>`

字段：

...

ID：证书持有者标识

VER：持有者公钥（方案前缀 RSA/ELG + 公钥串）

SIG：颁发者用其私钥对 `<b>dataForSign()</b>` 的签名

ID_TA：颁发者（上级 CA / 根 CA，自签时为自身）

FLAG1：签名算法（0=RSA，1=ElGamal）

FLAG2：用途（0=加密，1=签名）

...

- `<b>dataForSign() = ID + "|" + VER</b>`。
- `<b>issueCertificate</b>`：用 `<b>taPriv</b>` 对 `<b>dataForSign()</b>` 的 SHA-256 摘要签名得到 `<b>SIG</b>`。
- `<b>verifyCertificate(cert, issuerPub)</b>`：用颁发者公钥重新计算 `<b>H(dataForSign())</b>` 并验证 `<b>SIG</b>`。
- 文本格式：`<b>toTxt()</b>` 产出上述字段的键值文本，可 `<b>fromTxt()</b>` 还原，满足“证书以 txt 文本保存”。

3.5 简易 PKI

- `<b>CertRepo</b>`：单例 `<b>std::map<ID, Cert></b>` 证书库，提供 `<b>addCert</b>`/`<b>getCert</b>`/`<b>getCertPath</b>`/`<b>saveAll</b>`。
- `<b>getCertPath(subjectID)</b>`：从 `<b>subjectID</b>` 出发，沿 `<b>ID_TA</b>` 上溯到自签根 CA，返回链 `<b>[root, ..., subject]</b>`。
- `<b>verifyCertPath(path)</b>`：自根向下，逐张用颁发者公钥验证下一张证书。
- `<b>CA::initRoot</b>`：根 CA 自签名（`<b>ID_TA=自身</b>`，`<b>flag2=签名</b>`）。`<b>CA::initSub</b>`：下级 CA 由父 CA 签发。
- `<b>PKI::build</b>`：建库（`<b>clear</b>` 后）生成根 CA，再由其签发 CA1、CA2；调用 `<b>saveAll("certs")</b>` 落盘。
- 用户证书：`<b>Entity::requestCerts(CA&</b>` 向某下级 CA 申请“加密证书（`<b>id+"_ENC"</b>`，`flag2=0`）”与“签名证书（`<b>id+"_SIG"</b>`，`flag2=1`）”，存入证书库。

3.6 简易安全邮件系统

- `<b>sendMail(sender, receiver, msg, cipher, log)</b>`：
 1. 查询接收方加密证书链并验证；
 2. 用发送方签名私钥对 `<b>msg</b>` 摘要签名得 `<b>s</b>`；
 3. 组合 `<b>payload = msg + "\n@@SIG@@\n" + s</b>`，打包为 `<b>ZZ</b>`；
 4. 用接收方加密公钥 `<b>encryptMessage</b>` 得到 `<b>cipher</b>`。
- `<b>receiveMail(receiver, sender, cipher, msgOut, log)</b>`：
 1. 用接收方加密私钥 `<b>decryptMessage</b>` 还原 `<b>payload</b>`，按分隔符拆出 `<b>msg</b>` 与 `<b>s</b>`；
 2. 查询发送方签名证书链并验证；
 3. 用发送方签名公钥 `<b>verifyMessage</b>` 验证签名；通过则 `<b>msgOut=msg</b>`。

4. 测试与运行结果

编译：`<b>build.bat</b>`（或手动 `<b>g++ -O2 -std=c++11 -o CryptoCourse.exe *.cpp</b>`）。

通过菜单“8. 运行全部测试”得到（512 比特素数，1024 比特同理已单独验证）：

- `<b>RSA</b>`：加解密一致；签名/验证；密钥字符串往返一致。
- `<b>ElGamal</b>`：加解密一致；签名/验证；密钥字符串往返一致。
- `<b>证书方案</b>`：颁发证书（带 `<b>flag1=0, flag2=0</b>`）并验证通过；篡改 `<b>VER</b>` 后验证失败（防篡改）。
- `<b>PKI</b>`：证书库含 3 张 CA 证书；查询 `<b>Alice_ENC</b>` 证书链 `<b>CA_root ← CA1 ← Alice_ENC</b>`，证书链验证通过。
- `<b>安全邮件</b>`：发送端验证接收方加密证书链通过并加密；接收端验证发送方签名证书链 `<b>CA_root ← CA1 ← Alice_SIG</b>` 通过，签名验证通过，恢复明文与原文一致，邮件收发成功。

证书已落盘于 `<b>src/certs/</b>`（CA_root.txt、CA1.txt、CA2.txt 等）。

5. 使用说明

1. 双击 `build.bat` 编译（或手动用 g++ 编译全部 `.cpp`）。
2. 运行 `CryptoCourse.exe`，根据菜单选择：
 - 1 - 4：RSA / ElGamal 加解密与签名验证；
 - 5：证书方案（flag1/flag2）；
 - 6：PKI 证书链查询与验证；
 - 7：安全邮件收发演示；
 - 8：运行全部测试；
 - 9：切换密钥强度（512 / 1024，1024 完全符合任务书对 1024 比特素数的要求，耗时更长）。
3. 程序从文件/控制台读取明文与签名消息；大量输入建议以文件方式准备（演示默认内置示例消息）。

6. 不足与改进

- 素数为 1024 比特时，Miller – Rabin 轮数与模幂在大整数库下耗时较高；可改用以汇编优化的 GMP/NTL 替换内置 `ZZ` 以提速（接口已兼容）。
- 当前为教学演示，缺少前向安全、证书撤销列表（CRL）与有效期校验，可在 `Cert` 中追加 `NOTBEFORE/NOTAFTER` 字段并在 `verifyCertificate` 中校验。
- 安全邮件未做防重放/顺序号；可补充一次性随机数与时间戳。

附：源码目录 `CryptoCourse/src`、证书输出目录 `CryptoCourse/src/certs`、本说明所在 `CryptoCourse/doc`。